Finally, the `Cargos` message contains a list of `Cargo` achieved by the `repeated` notation. Internally, this is implemented by a `Seq[Cargo]` within a `Cargos` case class.

With this new .proto definition, you need a different implementation of the `ContainerServiceImpl`. This implementation needs not only the execution context to run futures but also a `ClusterSharding` reference to find the entities.

To add a cargo by sending it to the `Container` actor, you now have direct access to the `CargoEntity`; and with it, you can create a `Cargo` to send to the container:

```
class ContainerServiceImplSharding(
    sharding: ClusterSharding)(
    implicit val executionContext: ExecutionContext)
    extends ContainerService {

  override def addCargo(in: CargoEntity): Future[AddedCargo] = {          ❶
    val container =                                                       ❷
      sharding.entityRefFor(Container.TypeKey, in.entityId)

    container ! Container.AddCargo(Container.Cargo(in.kind, in.size))     ❸
    Future.successful(AddedCargo(true))                                   ❹
  }

  override def getCargos ...
}
```

❶ Has direct access to a CargoEntity Object

❷ Searches for the entity

❸ Converts the input of the service into the protocol of the actor and sends it

❹ Always responds AddedCargo(true)

Finally, let’s implement the second RPC call whose signature is implemented as follows: